I. Virtual Memory & Page Replacement (OSNo09) 🔥🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

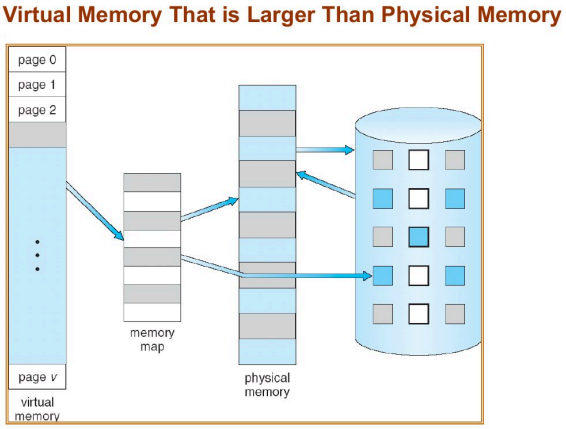
ความหมาย: Virtual Memory (หน่วยความจำเสมือน) = โปรแกรมไม่ต้องอยู่ใน RAM ครบทั้งตัว โหลดเฉพาะเพจที่กำลังใช้ (demand paging) ทำให้ RAM จริงเหมือนใหญ่กว่าจริง · Page Replacement = เมื่อ RAM เต็ม ต้องเลือกเพจ "ตัวปลอด" ออก

ความสำคัญกับข้อสอบ: นับ page fault (เพจยังไม่อยู่ใน RAM) ทั้ง FIFO (เข้าก่อน-ออกก่อน) / Optimal (ดีที่สุดเชิงทฤษฎี) / LRU (รู้ผล 15/9/12 ของ string หลัก), Belady's Anomaly (ปรากฏการณ์เพิ่ม RAM แล้ว fault เพิ่ม), Thrashing (สลับเพจหนักจนระบบแย่) — เป็นแนวคำนวณหลักอีกแนวของ Final

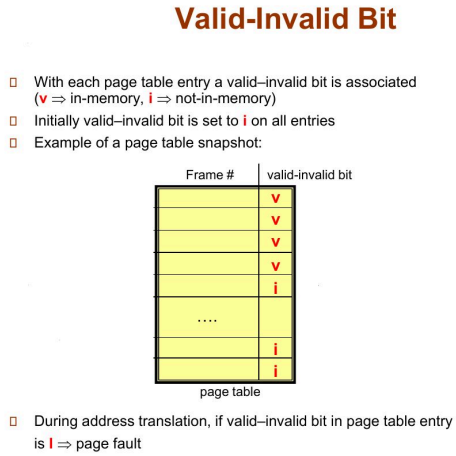
ใช้จริงในคอมพิวเตอร์: เปิดเกมใหญ่กว่า RAM เครื่องได้, สลับแอปเร็วโดยไม่โหลดใหม่, ขนาด "หน่วยความจำเสมือน" ที่เห็นใน Task Manager คือกลไกนี้ — ข้อมูลที่ไม่ได้ใช้ถูกส่งไปพักบน disk

I1. Demand Paging (โหลดเพจเฉพาะเมื่อถูกใช้) — ขอเมื่อไรค่อยเอามา

- **Virtual memory** = แยก logical memory ออกจาก physical → address space ใหญ่กว่า RAM จริงก็ได้ · implement ผ่าน **demand paging** หรือ demand segmentation
- **Demand paging**: โหลดเพจเฉพาะเมื่อถูกเรียกใช้ — หลักขี้เกียจของ **lazy swapper** (ตัวทำงานกับเพจเรียก **pager**) · ผลดี: I/O น้อย, RAM น้อย, ตอบเร็ว, รองรับ user มาก
- เพจที่ยังไม่โหลด → bit = **i (invalid)** → แปลง address เจอ i → **Page Fault (เพจยังไม่อยู่ใน RAM)** — **invalid ที่นี่ส่วนใหญ่ไม่ใช่ error** แค่ "ยังไม่มา"

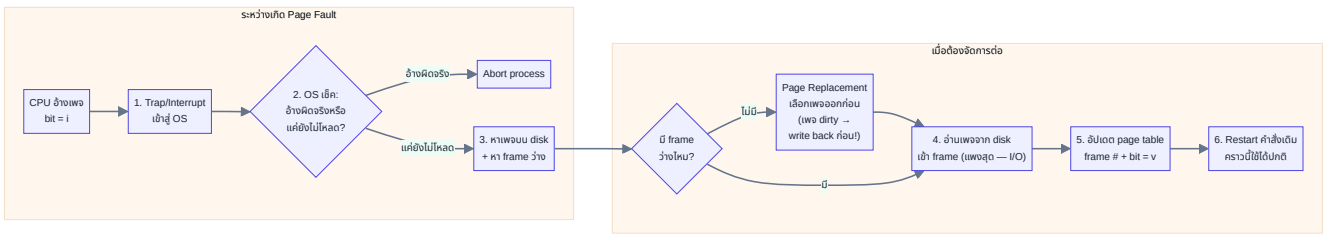


สไลด์ p.4 Virtual Memory ใหญ่กว่า Physical



สไลด์ p.6 Valid–Invalid Bit ใน page table

I2. Page Fault — ขั้นตอนจัดการ 6 ขั้น (เรียงได้ = ได้คะแนน)



1. Trap/Interrupt เข้าสู่ OSMMU เจอ bit = i (invalid) → ยิง trap ขึ้น kernel mode
2. OS ตรวจชนิดของ invalidอ้างเกิน address space จริง → abort · แค่ยังไม่โหลด → ไปขั้นถัดไป
3. หาเพจบน disk + หา frame ว่างถ้าไม่มี frame ว่าง → ใช้ Page Replacement ก่อน
4. อ่านเพจจาก disk เข้า frameขั้นที่แพงที่สุด (เป็น I/O ทั้งตัว)
5. อัปเดต page tableใส่ frame number + เปลี่ยน bit เป็น v (valid)

6 Restart คำสั่งเดิมCPU รัน instruction ที่พังอีกครั้ง — คราวนี้อ่านได้ปกติ

> ⚠️ **Dirty page**
> เพจที่ถูกแก้ไข (dirty (เพจที่ถูกแก้ไขข้อมูลแล้ว)/modified = 1) ต้อง**เขียนกลับ disk ก่อน** (write back (เขียนกลับลงดิสก์ก่อน)) ค่อยแทนที่ — เพจ clean (เพจที่ยังไม่ถูกแก้ไข (ดิสก์มีสำเนาแล้ว)) ทับได้เลยเพราะ disk มีสำเนา

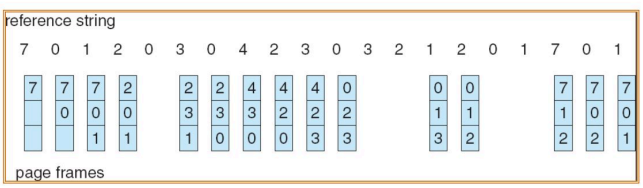
I3. Page Replacement — นับ fault 3 อัลกอริทึม 🔥🔥

โจทย์หลัก (สไลด์ใช้ string เดียวกัน, 3 frames (เฟรมหลายก้อน) เริ่มว่าง):

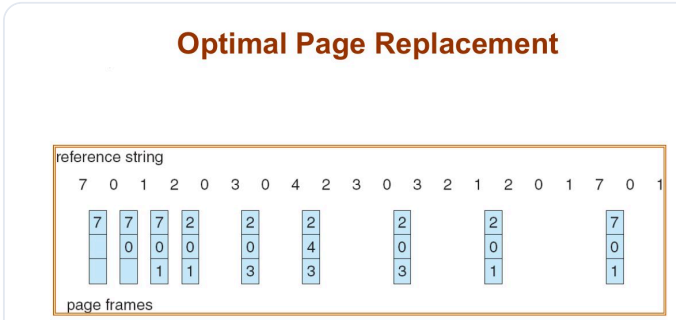
```
Reference String (ลำดับการอ้างเพจ): 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
```

อัลกอริทึม	มองไหน	หลักเลือก victim	Faults (3 frames)	ใช้จริง?
FIFO	คิวเข้า	เข้ามาก่อน ออกก่อน (ไม่สนว่าใครฮิต)	**15**	ง่ายแต่โง่ · เจอ Belady
Optimal (OPT)	อนาคต 👁️	ตัวที่**จะไม่ถูกใช้อีกนานสุด**ในอนาคต	**9**	**ใช้จริงไม่ได้** (รู้อนาคตไม่ได้) — ไว้เทียบ benchmark
LRU	อดีต 🔙	ตัวที่**ไม่ได้ใช้มานานสุด**	**12**	ใช้จริงได้ (เก็บประวัติย้อนหลัง) ดีกว่า FIFO

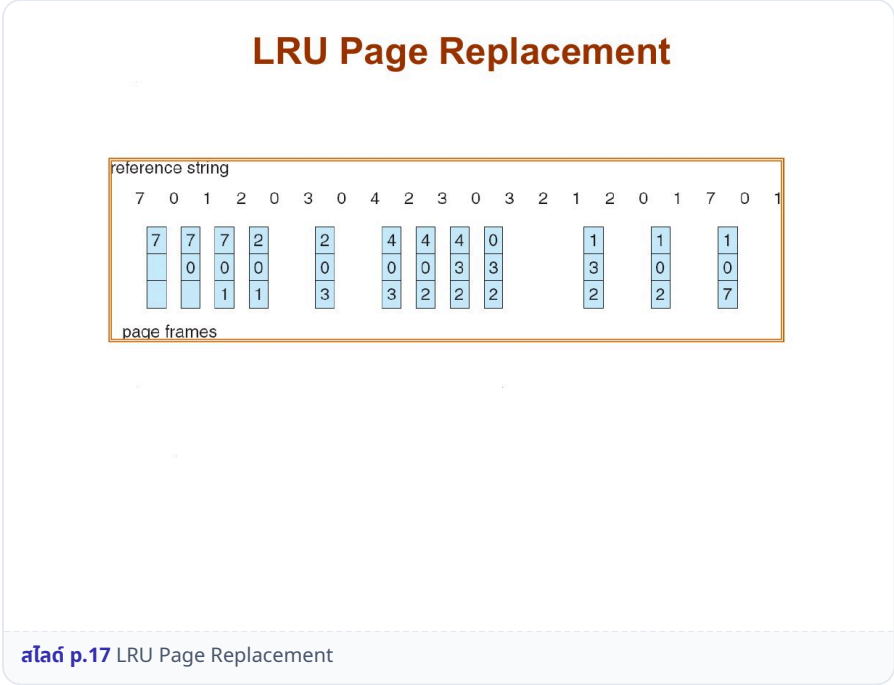
> 👕 **ตู้เสื้อผ้า 3 ชั้น**
> FIFO = รีดตัวใส่นานสุดก่อน · Optimal = หมอดู รีดตัวที่จะไม่ใส่อีกนานสุด · LRU = จำได้ว่าใส่ตัวไหนล่าสุดเมื่อไร รีดตัวที่ทิ้งไว้นานสุด



สไลด์ p.13 FIFO Page Replacement



สไลด์ p.16 Optimal Page Replacement



สไลด์ p.17 LRU Page Replacement

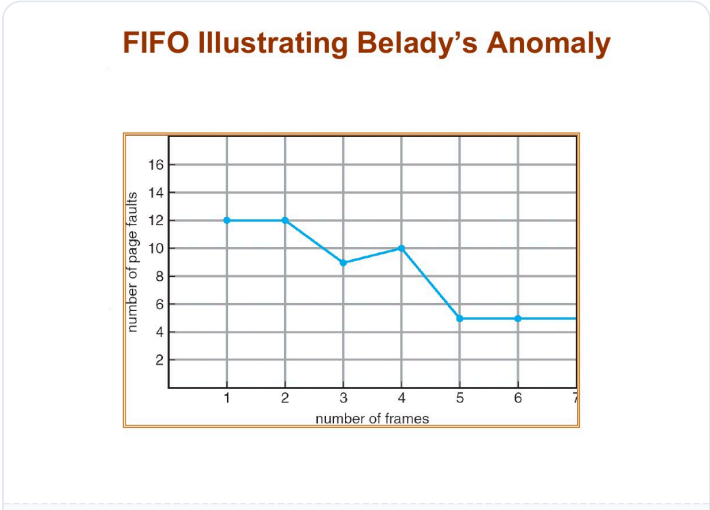
💡 เทคนิคไล่มือ: เขียน 3 ช่อง frame ใต้ string ทีละตัวอักษร · FIFO บอกตัวเลือก "เก่าสุด" ไว้ในหัว · LRU เขียนเลขกำกับ "ใช้ล่าสุดเมื่อไร" ใต้ช่อง · OPT กลับไปดูขวาของ string

I4. Belady's Anomaly — เพิ่ม frame แล้ว fault กลับโต! 🤯

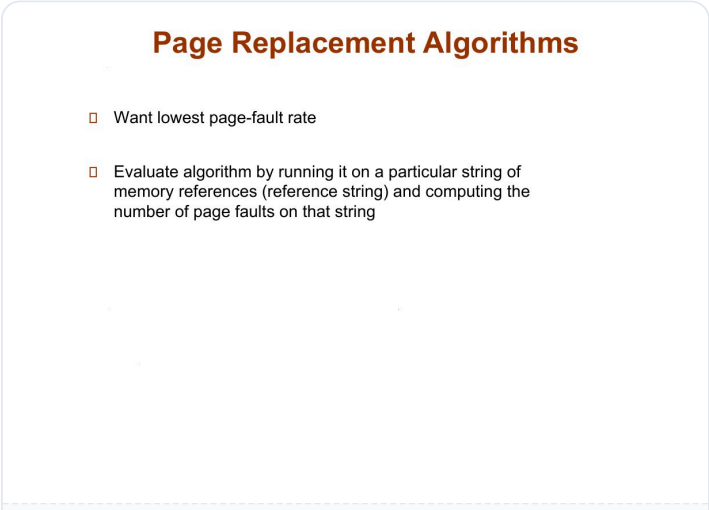
String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (FIFO):

จำนวน frame	3 frames	4 frames
Page faults (เหตุการณ์เพจไม่อยู่ใน RAM)	**9**	**10** 😱

เพิ่ม RAM fault กลับเพิ่ม — เกิดได้กับ **FIFO** (และ non-stack algorithm) แต่**ไม่เกิดกับ LRU / Optimal**



สไลด์ p.14 FIFO Illustrating Belady's Anomaly



สไลด์ p.11 Page Replacement Algorithms (กราฟ fault vs frames)

I5. EAT เมื่อมี Page Fault Rate (p)

`EAT = (1 − p) × memory access time + p × page fault service time`

ตัวเลขชวนคิด: mat = 200ns, fault service = 8ms → ที่ p = 0.001 (fault แค่ 0.1%) EAT = 0.999×200 + 0.001×8,000,000 ≈ 8,200ns = 8.2µs — ช้าขึ้น 40 เท่า! ห้ามให้ p โต

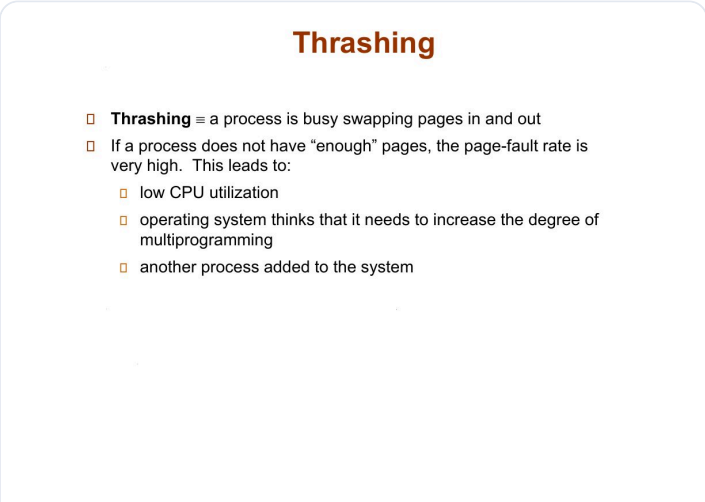
I6. Global vs Local Allocation

- **Global replacement**: แทนที่ได้จาก **frame ใดก็ได้ในระบบ** — แย่งข้าม process ได้
- **Local replacement**: เลือกจาก**เฉพาะ frame ของตัวเอง** (คนละบ้านกับคนละบ้าน)

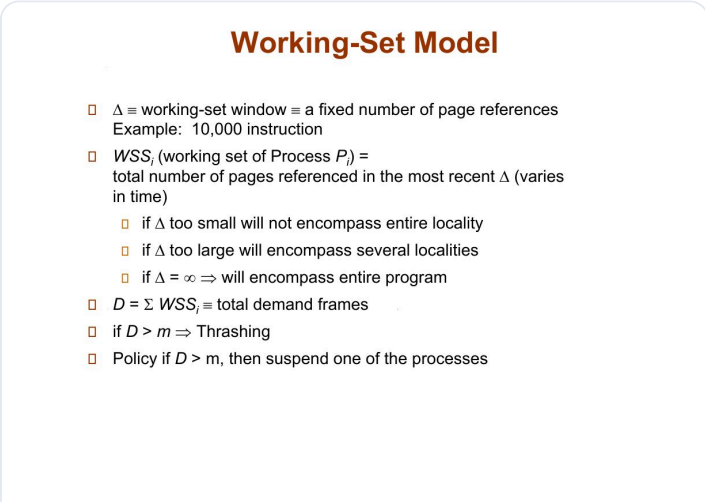
I7. Thrashing · Locality · Working Set (ชุดเพจที่ต้องมีใน RAM) 🔥

- **Thrashing**: process มีเพจไม่พอ → สลับเพจเข้า-ออก disk ตลอดเวลา → **CPU utilization ดิ่ง** · OS เข้าใจผิดว่า "งานน้อย" เลยเพิ่ม process → thrashing (สลับเพจหนักจนระบบแย่) ยิ่งหนัก (วงจรนรก) · เกิดทั้งข้ามงานและในงานเดียว (ลูปข้ามเพจ)
- **Locality of reference**: โปรแกรมมักอ้างกลุ่มเพจเดิมซ้ำ ๆ ในช่วงสั้น ๆ (อยู่ในฟังก์ชัน/ลูปเดิม)
- **Working set**: กลุ่มเพจที่ process ต้องมีใน RAM เพื่อไม่ให้ fault ในช่วง locality นั้น — มองย้อนหลัง Δ (working-set window เช่น 10,000 instruction) · **WSS_i** = จำนวนเพจที่ P_i อ้างใน Δ · Δ เล็กไปครอบ locality ไม่หมด / ใหญ่ไปกินหลาย locality / Δ = ∞ คือทั้งโปรแกรม
- **D = Σ WSS_i** = ความต้องการ frame รวม · ถ้า **D > m (frame จริง)** → **thrashing!** → OS ต้อง **suspend บาง process** คืน frame ให้คนที่เหลือ

> 🍳 **อุปมา**
> Working set = ของที่ต้องมีบนเคาน์เตอร์เพื่อทำเมนูให้จบ ถ้าวางไม่ครบ คุณวิ่งไปตู้เย็น (disk) ทุก 30 วินาที = ไม่มีอะไรเสร็จ → ทางแก้: ลดจำนวนเมนูที่ทำพร้อมกัน (suspend process)



สไลด์ p.19 Thrashing



สไลด์ p.22 Working-Set Model (D = ΣWSSᵢ, D > m → suspend)

I8. โจทย์สุดคลาสสิก: Array 128×128 🔥 (สไลด์ p.24)

`int data[128][128]` · **1 แถว = 1 เพจ** · เก็บแบบ **row-major** (เรียงตามแถว)

	ลูป	พฤติกรรม	Page faults
❌ **แย่**	`for (j...) for (i...) data[i][j]=0;` (column นอก, row ใน)	ทุกครั้งกระโดดข้ามแถว = เปลี่ยนเพจทุกครั้ง	**128 × 128 = 16,384**
✅ **ดี**	`for (i...) for (j...) data[i][j]=0;` (row นอก, column ใน)	อยู่เพจเดียวกันทั้งแถว fault เฉพาะตอนเปลี่ยนแถว	**128**

โค้ดเหมือนกัน สลับลูปเท่านั้น → `fault` ต่างกัน `128` เท่า `(16,384 : 128)`

บทเรียน: เขียนลูปให้เข้าถึงข้อมูลต่อเนื่องตาม row-major = locality ดี

ใส่ address จริงให้เห็นตา — ทำไมถึงต่างกันขนาดนั้น

สมมติเพจเริ่มที่ address `A` — สมาชิกติดกันในแถวห่างกัน 4 bytes แต่**คนละแถวห่างกัน 128 × 4 = 512 bytes = ครบ 1 เพจพอดี**

	ลำดับการอ้าง address	ผลต่อเพจ
✅ **row นอก** `data[0][0] → data[0][1] → …`	A, A+4, A+8, … , A+508 (128 ตัวต่อเนื่อง) → แล้วค่อยกระโดดไป A+512	เพจเดียวกันทั้งแถว → **fault 1 ครั้ง/แถว** × 128 แถว
❌ **column นอก** `data[0][0] → data[1][0] → …`	A, A+512, A+1024, … (ทุกครั้งข้าม 1 เพจเต็ม)	เปลี่ยนเพจ**ทุกครั้ง**ที่แตะสมาชิก → 16,384 ครั้ง

💡 หลักเดียวกันใช้ได้กับทุกโจทย์ลักษณะนี้: ดูว่าการอ้างสมาชิกตัวถัดไป "ห่างกันกี่ bytes" — ถ้าห่างน้อยกว่าขนาดเพจ = อยู่เพจเดียวกัน (ไม่ fault) · ถ้าห่างเกินขนาดเพจ = fault